

Développement web

Développement web

Publication date 2024

Table of Contents

1. Présentation	1
1. Objectifs	1
2. Environnement de travail	1
2. Apprentissage de PHP	4
1. Environnement d'exécution	4
2. Script, variables et typage dynamique	4
3. Fonctions et paramètres typés	6
4. POO en PHP	7
4.1. Une promotion d'étudiants	7
4.1.1. La classe Étudiant	7
4.1.2. Attribut statique, méthodes statiques	9
4.1.3. Collections, Exception	10
4.1.4. Héritage, classes abstraites	10
4.1.5. Interfaces, Trait	11
4.1.6. Espaces de nommage	12
4.1.7. Autochargement	12
4.2. Pour aller plus loin, quelques design pattern	13
4.2.1. Fabrique	13
4.2.2. Prototype	14
5. Le client serveur	14
5.1. Les requêtes	15
5.1.1. Exécuter un script sur le serveur	15
5.1.2. Utiliser les méthodes de requête HTTP	16
5.2. Les cookies et les sessions	17
5.2.1. La personnalisation	18
5.2.2. La gestion des sessions	19
6. Pour aller plus loin : Limitons les accès	20
3. Création d'un micro-framework	24
1. L'approche MVC	24
1.1. Le contrôleur frontal et l'inclusion des constantes	24
1.2. Le routage	25
1.3. Les contrôleurs	27
1.4. Le modèle	28
1.5. La vue	30
2. La persistance	30
2.1. Création et peuplement de la base	30
2.2. Création d'un repository avec une source de données	31
3. Faisons grandir l'application	33
3.1. Afficher un produit	33
3.2. Créer d'un produit	34
3.3. Modifier un produit	35
3.4. Supprimer un produit	35
4. Mise en oeuvre d'un framework (CI4)	36

List of Figures

1.1. Architecture d'un pod	2
2.1. PHP en client/serveur	4
2.2. Classe Etudiant	8
2.3. Héritage	10
2.4. Factory	13
2.5. Prototype	14
2.6. Exemple de requête	14
2.7. Les cookies HTTP	18
2.8. Cookie pour la personnalisation	19
2.9. Session	20
3.1. Notre MVC	24

Chapter 1. Présentation

1. Objectifs

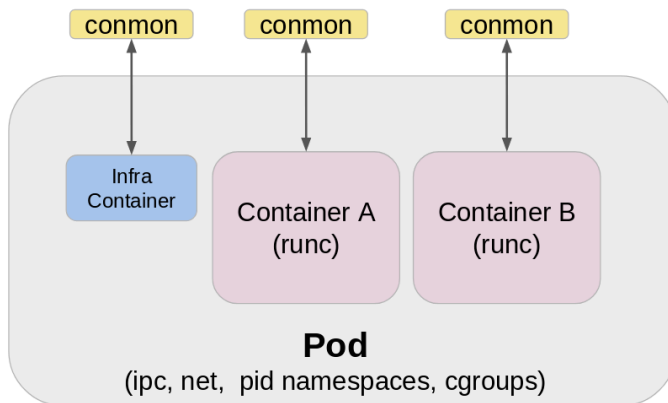
Dans la ressource R1.02:Développement d'interfaces web, vous avez appris à réaliser des pages web statiques. Dans cette ressource nous allons produire des pages Web dynamiques via un serveur HTTP (Hypertext Transfer Protocol). Le langage choisi est PHP (Hypertext Preprocessor), un langage de script impératif orienté objet. Pour la persistance de nos données nous utiliserons le SGBD (Système de Gestion de Base de Données) Postgress.

Nous disposons dans cette ressource de 5 créneaux de 1h20 de CM et de 22 créneaux de TD à raison de 2 par semaine. Voici une progression indicative :

Semaines	CM	TD	TD de SAE	SAE	Évaluation	Objectifs
S45	1h20	2h40				Apprentissage de PHP
S46	1h20	2h40				
S47	1h20	2h40			1h20	Évaluation (1h20)
S48	1h20	4h				Création d'un micro-frame-work
S49	1h20	8h				
S50		5h20			2h40	
S51			8h			Découverte de CodeIgniter 4
S52				5h40		TD de SAE
S53					SAE 2h40 (10 minutes écrit, 10 minutes présentation, 15 minutes questions)	

2. Environnement de travail

Nous allons utiliser des conteneurs légers, une méthode de virtualisation légère pour exécuter plusieurs environnements virtuels (conteneurs) simultanément sur un hôte unique. Nous aurons deux conteneurs l'un portant PHP/Apache et l'autre Postgress. Les deux conteneurs seront dans le même pod (un groupe de containers qui vont partager des informations entre eux).

Figure 1.1. Architecture d'un pod

Création du pod et des conteneurs depuis un terminal fedora (**exit** pour quitter ubuntu) :

1. création d'un pod nommé web-dev, qui exposera le port interne 80 sous le port 8080 : **podman pod create --name web-dev --publish 8080:80**
2. récupération de l'image de postgres : **podman pull postgres**
3. lancement du conteneur db avec comme user "devuser" et mot de passe "devuser" qui exécutera l'image postgres : **podman run --pod=web-dev --env POSTGRES_USER=devuser --env POSTGRES_PASSWORD=devuser --name db --detach postgres**
4. récupération de l'image de php8.3 et apache : **podman pull php:8.3-apache**
5. lancement du conteneur web qui exécutera l'image php:8.3-apache : **podman run --pod=web-dev --name web --detach php:8.3-apache**

En résumé :

```
//Creation du pod web-dev
podman pod create --name web-dev --publish 8080:80

//recup de postgres
podman pull postgres

//lancement de postgres
podman run --pod=web-dev --env POSTGRES_USER=devuser --env POSTGRES_PASSWORD=devuser \
--name db --detach postgres

//recup de php8.3 et apache
podman pull php:8.3-apache

//lancement de php8.3 et apahce
podman run --pod=web-dev --name web --detach php:8.3-apache
```

Une remarque sous les conteneurs, ils sont préconfigurés avec un compte root pour les services ce qui n'est pas une bonne pratique et ne sera pas à reproduire si vous utilisez une VM.

Quelques commandes utilises :

Les man

Les man sont accessibles via le "-" **man podman-pod, man podman-pod-list, ...**

Liste des pod

podman pod list

Arrêter, lancer, relancer, créer, supprimer un pod

podman pod start (ou stop ou restart ou create ou rm) nom_du_pod

Lister les conteneurs d'un pod

podman ps --filter pod=nom_du_pod

Comment accéder avec nos conteneurs vous pouvez utiliser **podman exec** pour exécuter des commandes par exemple pour ouvrir un bash sur le conteneur web **podman exec --tty --interactive web /bin/bash**. Pour copier des fichiers **podman cp**.

Il est également possible d'accéder directement à vos conteneurs depuis votre IDE, pour Visual Studio Code l'extension dev containers le permet, après installation sur >< choisir attache to a running conteneur, attention si vous êtes déjà connecté en ssh sur ubuntu (>ssh:ubuntu<) il vous faut vous déconnecter avant.

Sauvegarder vos conteneurs à la fin de la séance et les recréer à la séance suivante (<https://www.iut-nantes.univ-nantes.fr/etu/wiki/index.php/Podman.html>):

1. Exporter **podman export web -o web.tar** et **podman export postgres -o postgres.tar**
2. Importer **podman import web.tar web** et **podman import postgres.tar postgres**
3. Recréer le pod et lancer les conteneurs **podman pod create --name web-dev --publish 8080:80, podman run --pod=web-dev --env POSTGRES_USER=devuser --env POSTGRES_PASSWORD=devuser --name db --detach postgres, podman run --pod=web-dev --name web --detach php:8.3-apache.**

Chapter 2. Apprentissage de PHP

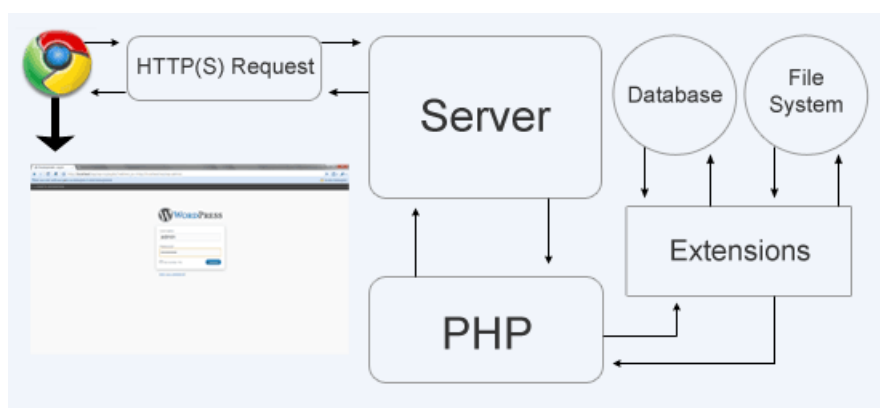
Nous allons apprendre PHP en construisant une application qui permet de commander des produits.

1. Environnement d'exécution

PHP peut-être utilisé en ligne de commande dans un terminal du conteneur web taper **php -i** vous observerez la configuration de PHP en ligne de commande (Command Line Interface). Nous reviendrons sur PHP en CLI lors de l'utilisation de frameworks pour des aides au développement.

Nous allons pour le moment utiliser PHP en mode client/serveur, une requête HTTP sera envoyé à un serveur HTTP qui en fonction de l'extension de la ressource demandée invoquera PHP.

Figure 2.1. PHP en client/serveur



Vous disposez d'un serveur HTTP dans le conteneur web, son répertoire de publication est `/var/www/html` déposer dans ce répertoire un fichier nommé `info.php` contenant votre premier script, les parties exécutées doivent être comprise dans la balise `<?php ?>`, `phpinfo()` est une fonction qui affiche la configuration de PHP.

```
<?php
phpinfo();
?>
```

Vous pouvez exécuter votre script avec l'URL suivante : `http://localhost:8080/info.php`. Par défaut, l'affichage des erreurs est activé (`display_errors ON`) mais quelles sont les types d'erreurs affichés (`error_reporting`), nous allons modifier la configuration de PHP pour afficher toutes les erreurs :

1. `phpinfo()` nous informe que le fichier de configuration `php.ini` est dans `/usr/local/etc/php` dans ce répertoire copier `php.ini-development` en `php.ini`
2. relancer le serveur apache `apachectl restart`
3. Observer que `error_reporting = E_ALL` (<https://www.php.net/manual/en/errorfunc.constants.php>)

2. Script, variables et typage dynamique

Vous trouverez la documentation ici <https://www.php.net/manual/fr/language.types.php>.

En PHP les variables sont typées dynamiquement à l'exécution et un script peut exécuter un autre script (`require`, `require_once`, `include`, `include_once`). Exécuter et comprendre les scripts suivants (vous pouvez copier le workspace fourni sur git avec **podman cp workspace web:/var/www/html**)

```
<!-- 01-script.php -->
```



```
<pre>
Hors du php
<?php
//echo permet un affichage
    echo "Dans du PHP ";
?>
Hors du php
<?php
    echo "de retour en PHP";

    //
    $user = "jub";
    require "02-script.php";
    echo $user;

?>
</pre>
```

```
<!-- 02-script.php -->
<?php
    echo '<pre>';
    echo $user;
    echo '</pre>';
    $user = ucwords($user);
?>
```

Le script suivant illustre la capacité d'une variable à changer de type à l'exécution, à être définie, indéfinie :

```
<pre>
<?php
    //la variable $a a t elle été déclarée
    //et est elle différente de null
    //var_dump affichage pour débbugage et isset pour tester
    var_dump(isset($a));

    //déclaration de $a et affectation de 10
    // $a est un int
    $a = 10;
    var_dump($a);

    // $a est maintenant de type null et vaut null
    $a = null;
    var_dump($a);

    // $a est maintenant un tableau association
    $a=array();
    var_dump($a);

    //ajout d'un élément au tableau
    $a['clef']='une valeur';
    var_dump($a);
    //ajout d'un élément au tableau
    $a[0]=10;
    var_dump($a);

    // $a n'est plus définie
    unset($a);
    var_dump(isset($a));
?>
</pre>
```

Les variables ont une portée (un scope) : global, local et static. Le script suivant illustre la portée globale :

```
<?php
// $a est global
$a = "abc";
//La variable magique $GLOBALS est un tableau associatif qui contient
//les variables globales
echo $GLOBALS['a'].'<br>';
```

```
function demo1() {
    //affiche $a si il est défini sinon le message non défini
    echo ($a?"non définie").'<br>';
}

function demo2() {
    //Le mot clef global permet d'accéder à une variable globale
    global $a;
    //affiche $a si il est défini sinon le message non défini
    echo ($a?"non définie").'<br>';
}
demo1();
demo2();
?>
```

Le script suivant illustre la portée locale :

```
<?php
//$a est global
$a = "abc";

function demo() {
    //Le scope de $a est ici local
    $a = 10;
    //affiche $a si il est défini sinon le message non défini
    echo ($a?"non définie").'<br>';
}

demo();
?>
```

Enfin, ce dernier script illustre la portée static (la variable continue à exister après l'appel du sous-programme :

```
<?php
function demo() {
    static $a = 0;
    echo ++$a.'<br>';
}

for ($i=0; $i<5; $i++)
    demo();
?>
```

3. Fonctions et paramètres typés

Si il n'est pas possible de définir le type d'une variable à la déclaration, il est possible de typer les paramètres d'une fonction ou d'une méthode. Vous trouverez ici la description des types : <https://www.php.net/manual/fr/language.types.type-system.php>. Les types peuvent être composites : union (|), intersection (&) et disposent de deux alias mixed (object|resource|array|string|float|int|bool|null) et iterable (Traversable|array).

Le passage d'argument et par défaut par valeur, le passage par référence est possible avec &, les arguments peuvent avoir des valeurs par défaut et être nommés, le nombre d'argument peut également être variable (...). Le programme suivant illustre un calcul de moyenne sur un tableau :

```
<?php
//avec une boucle for
function moyenne1(Array $tab=array(10)):Float {
    $somme = 0;
    foreach ($tab as $elt)
        $somme += $elt;
    if (count($tab)>0)
        $somme /= count($tab);
}
```

```
        return $somme;
    }
    //avec reduce et fonction anonyme
    function moyenne2(Array $tab=array(10)):Float {
        $somme = array_reduce($tab,function($carry, $elt){
            return $carry + $elt;}, 0.0);

        if (count($tab)>0)
            $somme /= count($tab);
        return $somme;
    }

    echo moyenne1(array(10,12))."<br>".moyenne2(array(10,12))."<br>";
    echo moyenne1(array()).'<br>'.moyenne2(array()).'<br>';
    echo moyenne1().'<br>'.moyenne2().'<br>';
    ?>
```

Écrire et tester une fonction qui prend deux String ou deux Float ou deux Int et qui renvoie le plus grand.

Écrire une fonction qui prend un tableau en paramètre et renvoie le tableau trié, le tableau en paramètre ne doit pas être modifié, vous pouvez utiliser sort (<https://www.php.net/manual/en/function.sort.php>).

Écrire une fonction qui prend un tableau en paramètre et le trie.

4. POO en PHP

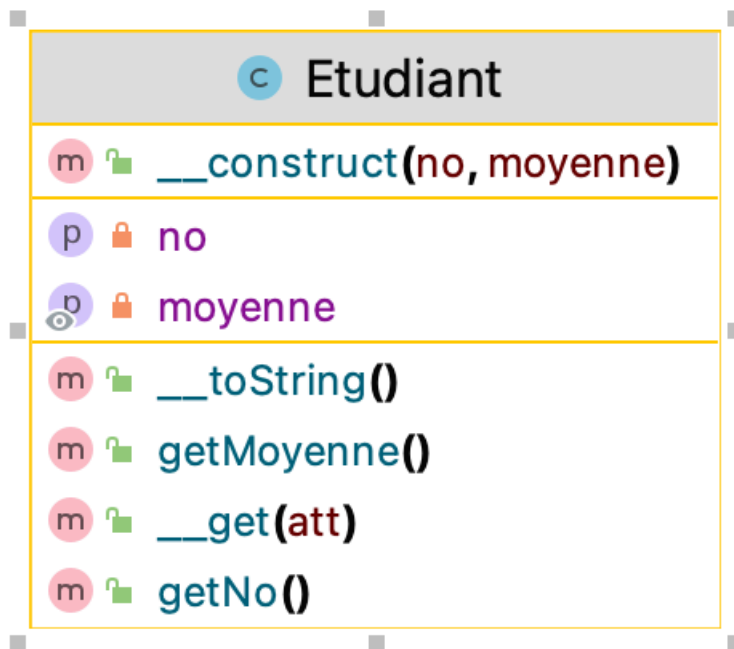
Vous trouverez ici les informations : <https://www.php.net/manual/fr/language.oop5.php>. Vous travailler dans le répertoire de publication web (/var/www/html).

4.1. Une promotion d'étudiants

4.1.1. La classe Étudiant

Créer une classe Etudiant (dans le script Etudiant.php) avec deux attributs privés no une string et une moyenne un flottant, les méthode getNo() et getMoyenne() permettront de consulter les attributs, un constructeur permettra de construire un étudiant avec un no et une moyenne choisis, si la moyenne n'est pas fournie, elle sera supposée égale à 10.

Figure 2.2. Classe Etudiant



Un accès direct en lecture seul à l'attribut "moyenne" sera possible. L'accès aux attribut privés conduiront au message "access denied", vous devrez passer par la méthode magique `__get($attr)`

```

public function __get($att){
    if ($att != "moyenne")
        return "access denied";
    else
        return self::getMoyenne();
}
  
```

PHP dispose également de `readonly` sur les attributs.

Le script suivant doit fonctionner :

```

<?php require "Etudiant.php"; ?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <title>Test Etudiant </title>
</head>
<body>
<pre>
<?php
    $etudiant1 = new Etudiant("E001",10);
    var_dump($etudiant1);
  
```

```

$etudiant2 = new Etudiant("E002",moyenne: 20);
var_dump($etudiant2);

echo $etudiant1->getNo()." : ".$etudiant1->getMoyenne();
echo '<br>';
echo "{$etudiant2->getNo()} : {$etudiant2->getMoyenne()}";

echo '<br>';
echo $etudiant1->moyenne;
echo '<br>';
echo $etudiant1->no;
echo '<br>';
echo $etudiant2;
echo '<br>';
$etudiant3 = new Etudiant("E002",moyenne: 20);
var_dump($etudiant3 === $etudiant2);
var_dump($etudiant3 == $etudiant2);
$etudiant4 = $etudiant3;
var_dump($etudiant4 === $etudiant3);
var_dump($etudiant4 == $etudiant3);
?>
</pre>
</body>

```

Vous aurez besoin du constructeur `__construct`, des méthodes magiques `__get()` et `__toString()`.

4.1.2. Attribut statique, méthodes statiques

Les informations se trouvent ici : <https://www.php.net/manual/en/language.enumerations.backed.php>, <https://www.php.net/manual/fr/language.oop5.static.php>.

Modifier votre classe étudiante pour quelle dispose d'un état mental partagé par tous les étudiants :

```

<?php
enum EtatMental : string
{
    case Heureux = 'Etudiant Heureux';
    case Joyeux = 'Etudiant Joyeux';
    case Bien = 'Etudiant Bien';
}
?>

```

Votre classe devra également pouvoir fournir le nombre d'étudiants déjà créé. Une variable statique sera incrémentée dans le constructeur.

Enfin, le programme suivant doit vous permettre de tester :

```

<?php require "Etudiant.php"; ?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <title>Test Etudiant 2 </title>
</head>
<body>
<pre>
<?php
    $etudiant1 = new Etudiant("E001",10);
    echo $etudiant1;
    $etudiant2 = new Etudiant("E002",moyenne: 20);
    echo '<br>';
    echo $etudiant2;
    echo '<br>';
    var_dump(Etudiant::$EtatMental);
    Etudiant::$EtatMental = EtatMental::Joyeux;
    echo '<br>';
    echo $etudiant2;
    echo '<br>';
    echo Etudiant::getNbEtudiant();

```

```
?>
</pre>
</body>
```

4.1.3. Collections, Exception

PHP dispose de collections mais dans cet exercice nous allons utiliser de simples tableaux : <https://www.php.net/manual/fr/book.array.php>. En PHP, les tableaux peuvent-être indexés ou associatifs.

Implémenter et tester une promotion d'étudiants, un étudiant caractérisé par son numéro ne pourra être ajouté deux fois cela devra déclencher une Exception. De même, la suppression d'un étudiant non membre lèvera une Exception. Vous devrez également fournir la moyenne de la promotion si elle n'est pas vide et une Exception sinon.

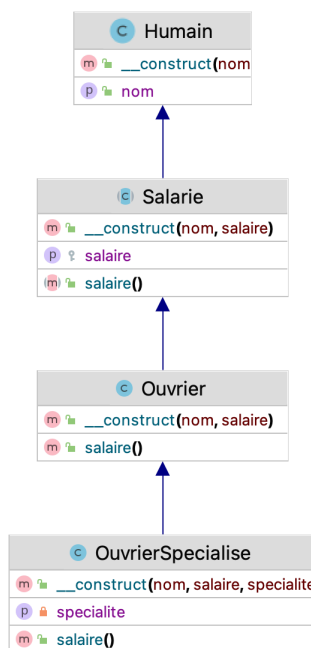
Si vous utilisez des tableaux indexés, vous pouvez avoir besoin de la boucle foreach; <https://www.php.net/manual/fr/control-structures.foreach.php> et de la fonction array_splice : <https://www.php.net/manual/en/function.array-slice.php>.

Si vous utilisez des tableaux associatifs, la méthode array_key_exists peut vous aider : <https://www.php.net/manual/en/function.array-key-exists.php> et de la fonction unset : <https://www.php.net/manual/fr/function.unset.php>.

4.1.4. Héritage, classes abstraites

Implémenter le diagramme de classe suivant :

Figure 2.3. Héritage



La méthode `salaire()` permet de connaître le salaire :

- Pour un salarié, il ne peut être calculé
- Pour un ouvrier c'est le salaire
- Pour un ouvrier spécialisé, c'est le salaire d'un ouvrier majoré 10%

L'utilisation de `#[\Override]` augmentera la robustesse de votre code. Le code suivant vous permet de tester la création d'un Humain à vous de faire les autres :

```
<?php
require "Humain.php";
$h = new Humain();
```

```
$h->nom="Joe";  
var_dump($h);  
?>
```

4.1.5. Interfaces, Trait

PHP dispose l'interface `IteratorAggregate` (<https://www.php.net/manual/en/class.iteratoraggregate.php>), rendre la collection de la classe suivante `Iterable` :

```
class Mots  
{  
    private $mots = [];  
  
    public function getItems()  
    {  
        return $this->mots;  
    }  
  
    public function addItem($mot)  
    {  
        $this->mots[] = $mot;  
    }  
}
```

Vous pourrez utiliser `ArrayIterator` <https://www.php.net/manual/en/class.arrayiterator.php>.

Le programme suivant doit fonctionner :

```
<?php  
require "Mots.php";  
$collection = new Mots();  
$collection->addItem("First");  
$collection->addItem("Second");  
$collection->addItem("Third");  
$collection->addItem("Fourth");  
  
foreach ($collection->getIterator() as $item) {  
    echo $item . "\n";  
}  
?>
```

Il n'est possible d'hériter que d'une classe et les interfaces ne disposent pas de code par défaut en PHP, les traits permettent de contourner cette limitation : <https://www.php.net/manual/fr/language.oop5.traits.php>.

Exécuter et comprendre le code suivant :

```
<?php  
  
trait school {  
    public function learn() {  
        echo "I am learning PHP at WayToLearnX! \n";  
    }  
}  
trait company {  
    public function work() {  
        echo "I am working with PHP at WayToLearnX!";  
    }  
}  
  
class Person {  
    use school, company;  
}  
$person = new Person();  
$person->learn();  
$person->work();  
?>
```

Il est possible d'utiliser plusieurs traits et de résoudre les conflits (`insteadof`).

4.1.6. Espaces de nommage

Les espaces de nommage ou espace de noms (namespace) désigne en informatique un lieu abstrait conçu pour accueillir des ensembles de termes appartenant à un même répertoire. Vous trouverez la documentation PHP ici <https://www.php.net/manual/en/language.namespaces.rationale.php>.

Le programme suivant est fonctionnel, si vous avez défini les bons répertoires et les bon espaces de nommage :

```
<?php
    require "un/A.php";
    require "deux/A.php";

    echo (new un\A())->a; //un
    echo (new deux\A())->a; //deux

    use un\A;
    echo (new A())->a; //un

    use deux\A as AA;
    echo (new AA())->a; //deux

?>
```

Le code des classes A est :

```
class A
{
    public $a="deux";
}
et
class A
{
    public $a="un";
}
```

.

4.1.7. Autochargement

Les require, include, require_once, include_once peuvent-être évités pour le chargement des classes.

PHP est un langage interprété que donne le code suivant et pourquoi :

```
<?php
    $a = "toto";
    $b="a";

    echo $$b;
```

La fonction spl_autoload_register se déclenche à l'exécution lorsqu'une classe, un trait ou une interface sont rencontrés : <https://www.php.net/manual/fr/language.oop5.autoload.php>.

Tester et comprendre le code suivant :

```
spl_autoload_register(function ($classe){
    echo $classe;die();
});

$obj1 = new un\Chien();
```

Dans le répertoire A placer la classe

```
namespace A;

class A
{
    function __construct() {
        echo "A";
    }
}
```



```
}
}
```

Dans le répertoire B placer la classe

```
namespace B;

class B
{
function __construct() {
    echo "B";
}
}
```

Dans le répertoire contenant les répertoires A et B modifier le code suivant pour qu'il fonctionne :

```
<?php

spl_autoload_register(function ($classe){
    $path=$classe;
    if (DIRECTORY_SEPARATOR === '/') {
        $path=str_replace('\\', '/', $classe);
    }
    //TODO
});
$a = new A\A();
$b = new B\B();
```

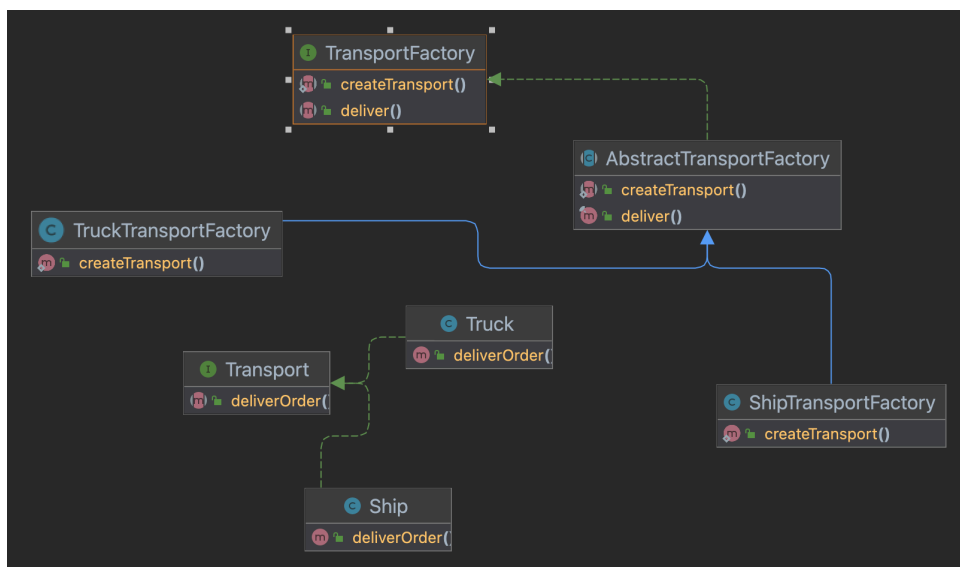
4.2. Pour aller plus loin, quelques design pattern

Le site suivant décrit des design patterns en PHP : <https://refactoring.guru/fr/design-patterns/php>.

4.2.1. Fabrique

Nous souhaitons créer une factory qui permet d'avoir des transports avec une méthode deliver au comportement spécifique :

Figure 2.4. Factory



Implémenter le design pattern factory en utilisant le diagramme de classe précédent, sachant que le code de la méthode deliver() de AbstractTransportFactory est :

```
final public function deliver()
{
    return $this->createTransport()->deliverOrder();
}
```

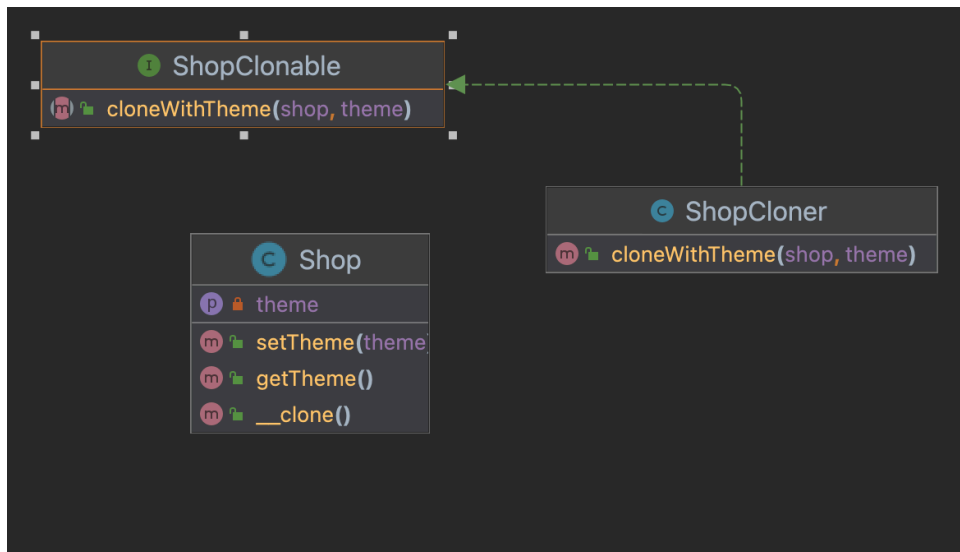
4.2.2. Prototype

La création d'un objet peut-être longue, il est parfois préférable de le recopier. Le design pattern Prototype (aussi appelé "Clone") consiste à permettre la copie d'un objet sans créer de dépendances fortes entre l'original et sa copie. Implémenter le design pattern prototype en utilisant le diagramme de classe précédent, sachant que le code de la méthode `__clone()` de Shop est :

```
public function __clone()
{
    $this->theme = clone $this->theme;
    //Sachant que le clone reste au premier niveau
}
```

La méthode `cloneWithTheme` clone le Shop et modifie son thème.

Figure 2.5. Prototype

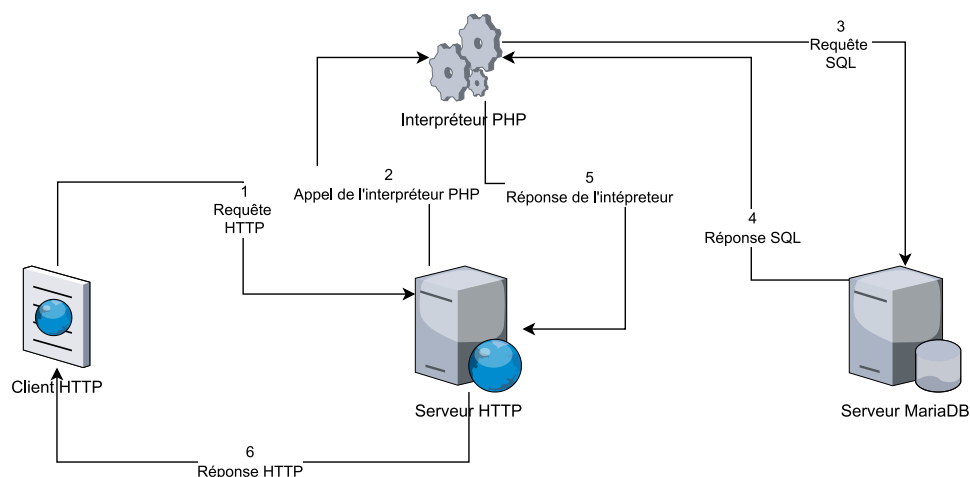


5. Le client serveur

Les protocoles ou environnement client serveur sont un mode de transaction où le client envoie une requête pour obtenir une réponse du serveur. On dit que le serveur offre des services qui sont sollicités par les clients. Le serveur pouvant lui-même être client d'un autre serveur.

Nous aurons un serveur HTTP, un serveur SQL (Structured Query Language) et un interpréteur PHP.

Figure 2.6. Exemple de requête



1. Le client HTTP effectue une requête en spécifiant une URL ;
2. Le serveur HTTP en fonction de l'extension de fichier renvoie directement la page (.htm, .html) ou passe la main à l'interpréteur PHP (.php) ; l'interpréteur PHP interprète les scripts PHP identifiés par la balise `<?php //TODO ?>`, la balise peut apparaître plusieurs fois dans un même fichier mais l'interprétation est unique ;
3. si le script fait appel à un serveur SQL alors une requête SQL est envoyée au serveur SQL ;
4. la réponse du serveur SQL est renvoyée par le serveur SQL puis utilisée par l'interpréteur PHP ;
5. l'interpréteur fournit le fichier créé au serveur HTTP ;
6. La réponse est enfin fournie au client HTTP.

Comme vous l'observez vous allez mettre en œuvre vos acquis du BUT1, HTML/CSS, SQL, réseaux. PHP étant un langage orienté objet vous devez également mobiliser vos compétences dans ce domaine.

5.1. Les requêtes

Pour chaque exercice, vous aurez à consulter la documentation officielle PHP : <https://www.php.net/docs.php>.

5.1.1. Exécuter un script sur le serveur

Créer un fichier avec l'extension .php contenant un script (`<?php phpinfo(); ?>`) qui lui-même utilise la fonction `phpinfo()`¹, déposer ce fichier sur votre espace de publication et déclenchez l'exécution du script via votre navigateur.

Répondez aux questions suivantes :

- Quelle est la version de PHP ?
- Dans les protocoles, les requêtes contiennent une entête et un corps, vous pouvez observer à travers votre navigateur l'entête envoyée par votre client
 1. clic droit :inspecter,
 2. network,
 3. rafraîchir,
 4. sélectionner la requête,
 5. header

Où pouvez-vous retrouver les informations du header HTTP dans le résultat de l'exécution du script ?

- Les interpréteurs PHP disposent d'un fichier de configuration (php.ini) qui limite les usages. Ici quelle est la taille maximum d'upload d'un fichier ?
- L'interpréteur PHP peut communiquer avec son environnement et par exemple obtenir des informations depuis le serveur HTTP au travers du tableau `$_SERVER`. Une URL HTTP peut contenir des fragments et parmi ses fragments des paramètres peuvent être passés, les paramètres sont introduits par `?`, ils sont sous la forme `clef=valeur` et sont séparés par `&`. Dans cet exemple `http://www.exemple.com:80/chemin/vers/monfichier.html?clé1=valeur1&clé2=valeur2#QuelquePartDansLeDocument` les paramètres sont `clé1` et `clé2`. Modifier l'URL appelée pour passer en paramètres `code_département` avec comme valeur 44, ville avec comme valeur Nantes et département avec comme valeur Loire Atlantique, que contiennent `$_SERVER['REQUEST_METHOD']`, `$_SERVER['QUERY_STRING']` ? Pourquoi l'espace a-t-il été transformé en `%20` ?

¹<https://www.php.net/manual/fr/function.phpinfo.php>

5.1.2. Utiliser les méthodes de requête HTTP

Le protocole HTTP reconnaît des méthodes de requête², nous utiliserons dans ce cours :

GET

pour récupérer des données

POST

pour envoyer des informations.

Nous utiliserons deux clients : votre navigateur et **curl** un client en ligne de commande. D'autres clients comme postman et Insomnia REST Client existent, nous les utiliserons le semestre prochain.

5.1.2.1. la méthode GET

Le trafic en GET est principalement généré avec la barre d'URL, les ancres et les formulaires. En PHP, les paramètres sont récupérés avec le tableau \$_GET³.

PHP est un langage interprété faiblement typé le type est inféré à l'exécution et une variable peut changer de type⁴.

Pour afficher sur la sortie standard PHP dispose de la fonction echo⁵, de la fonction print⁶ et de la fonction var_dump⁷.

En PHP, les tableaux sont associatifs⁸.

5.1.2.1.1. Afficher les paramètres

Nous utiliserons comme fragment de paramètres *?pseudo=laurent&password=pass&statut=admin*.

Écrire un script get1.php qui utilise \$_GET et var_dump pour produire l'affichage suivant :

```
array(3) { ["pseudo"]=> string(7) "laurent" ["login"]=> string(4) "pass"
["status"]=> string(5) "admin" }
```

Dans un fichier PHP, cohabitent du texte et des scripts, entourer votre script avec les balises <pre> </pre>, vous devriez obtenir

```
array(3) {
    ["pseudo"]=>
    string(7) "laurent"
    ["password"]=>
    string(4) "pass"
    ["status"]=>
    string(5) "admin"
}
```

var_dump est utilisée pour le débogage, en utilisant echo et \$_GET["clef"] écrire un script get2.php qui permet d'avoir la page HTML 5 suivante :

```
pseudo : laurent
password : pass
status : admin
```

²<https://developer.mozilla.org/fr/docs/Web/HTTP/Methods>

³<https://www.php.net/manual/fr/reserved.variables.get.php>

⁴<https://www.php.net/manual/fr/language.variables.variable.php>

⁵<https://www.php.net/manual/fr/function.echo.php>

⁶<https://www.php.net/manual/fr/function.print.php>

⁷<https://www.php.net/manual/fr/function.var-dump.php>

⁸<https://www.php.net/manual/fr/book.array.php>

Pour rappel une page HTML a la structure suivante :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Test2</title>
</head>
<body>
  ...
  <!-- Le reste du contenu -->
  ...
</body>
</html>
```

Nous avons utilisé trois fois `$_GET`, nous pouvons améliorer notre script avec la boucle `foreach`⁹. Une syntaxe alternative est également possible avec `<?php foreach ($tab as $clef=>$valeur): ?> <?php endforeach;?>`.

5.1.2.1.2. Envoyer des paramètres

Nous avons déjà envoyé des paramètres avec la barre d'URL, nous pouvons également utiliser le client en ligne de commande **curl**, vous pouvez obtenir la documentation de **curl** avec `man curl`. Tester et comprendre la commande `curl -v --noproxy "*" -X GET "http://localhost:8080/rep/get2.php?pseudo=laurent&login=pass&statut=admin"`.

Une autre solution est de créer un formulaire¹⁰, créer un formulaire `get2_form.html` qui permet de saisir le pseudo et le mot de passe, le statut sera envoyé avec un input de type hidden.

Créer un document HTML 5 qui contient 2 ancres qui référencent le même script `get2.php` dont chacune code un statut différent, ici nous passons dans l'URL les informations pour le serveur.

```
je suis admin
je suis visiteur
```

5.1.2.2. la méthode POST

Il est impossible de générer un trafic en POST depuis la barre d'URL, en effet les paramètres ne sont plus passés dans l'entête de la requête mais dans son corps. Il nous faudra un formulaire ou un langage comme javascript pour générer du POST depuis un navigateur.

Créer le fichier `get_post.php` qui devra afficher les paramètres comme le faisait `get2.php`, ce script devra fonctionner que la requête soit en GET ou en Évaluation. Vous aurez à utiliser `isset`¹¹ qui permet de savoir si une variable est déclarée et est non nulle, vous aurez également à utiliser une conditionnelle¹². Attention en PHP `$_GET` est toujours déclaré car en vérité il ne teste pas la méthode HTTP mais la présence de paramètres dans le header.

Vous pourrez tester votre code avec **curl** : `curl -v --noproxy "*" -d "pseudo=laurent&login=pass&statut=admin" -H "Content-Type: application/x-www-form-urlencoded" -X POST "http://serveur/~user/chemin/get_post.php"`.

Pour finir créer un formulaire et tester.

5.2. Les cookies et les sessions

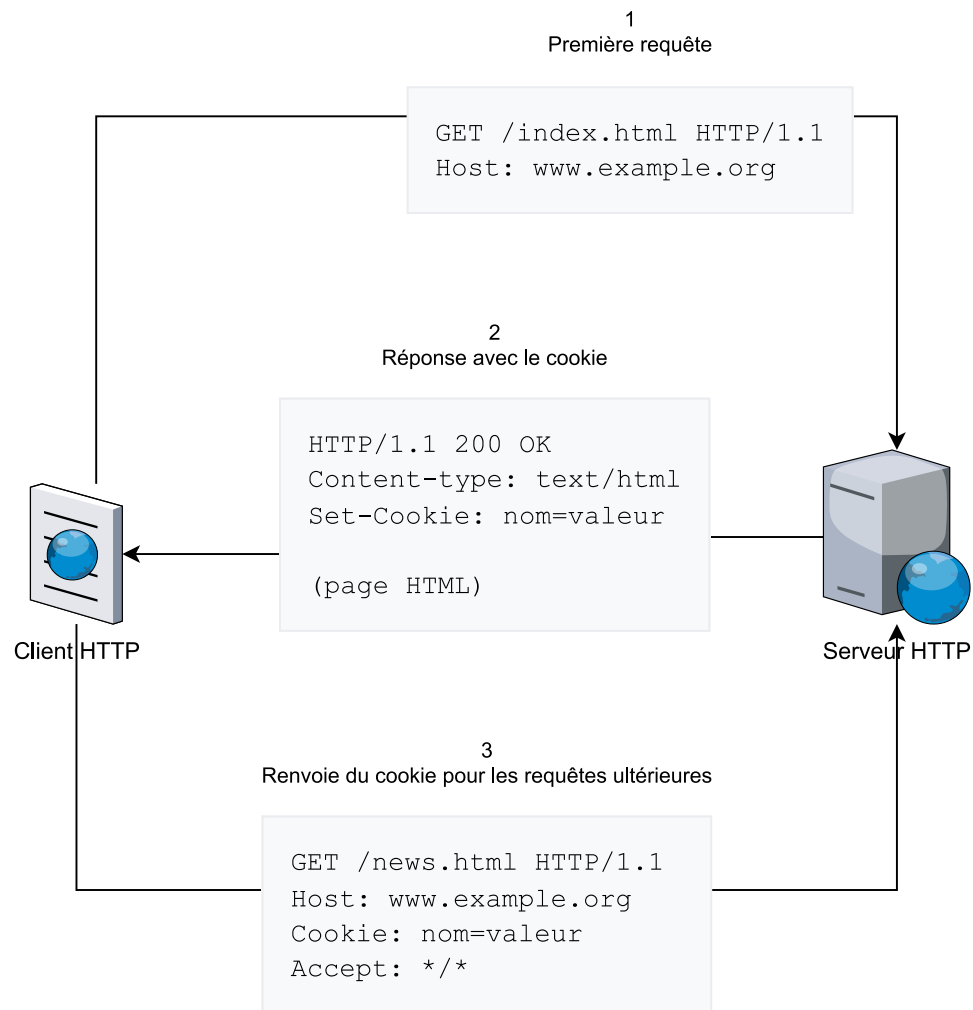
Le protocole HTTP est protocole sans état qui n'enregistre pas l'état de session lors d'une communication entre deux requêtes successives. Cependant le protocole HTTP permet de remédier à ce problème en utilisant des cookies. Un cookie est texte court envoyé par un serveur HTTP à un client HTTP, que ce dernier renverra les prochaines fois qu'il se connectera aux serveurs partageant le même nom de domaine.

⁹<https://www.php.net/manual/fr/control-structures.foreach.php>

¹⁰<https://developer.mozilla.org/fr/docs/Web/HTML/Element/Form>

¹¹<https://www.php.net/manual/fr/function.isset.php>

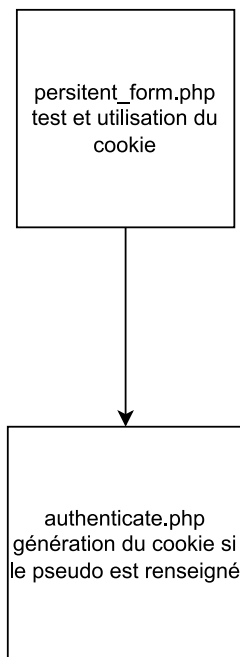
¹²<https://www.php.net/manual/fr/control-structures.if.php>

Figure 2.7. Les cookies HTTP

Les cookies sont utilisés pour la personnalisation, la gestion des sessions, le pistage, ...

5.2.1. La personnalisation

Depuis HTML5 les données peuvent-être validées par le client et la mémoire conservée dans le navigateur, les mêmes fonctionnalités peuvent être offertes par PHP. Nous allons créer un formulaire `persistant_form.php` qui demandera le pseudonyme et le mot de passe, si ce formulaire est consulté à nouveau alors le pseudonyme doit rester pré-rempli, ce formulaire aura pour cible `authenticate.php` qui créera le cookie.

Figure 2.8. Cookie pour la personnalisation

Le création d'un cookie est réalisée avec `setcookie()`¹³. Attention, selon la configuration du serveur, la création du cookie comme toute manipulation de l'entête HTTP doit avoir lieu avant toute écriture sur la sortie standard. La lecture est réalisée avec le table `$_COOKIE`¹⁴.

Quelques conseils :

- utiliser `htmlentities()`¹⁵ pour le contenu de votre cookie;
- dans votre navigateur, vous pouvez observer et supprimer les cookies stockés.

5.2.2. La gestion des sessions

Nous allons augmenter améliorer notre système.

5.2.2.1. Effectuer une direction

Dans `authenticate.php` si l'authentification échoue alors une direction est effectuée vers `persitent_form.php` sinon ok doit-être affiché. La direction en HTTP est en trois étapes :¹⁶

1. Le client demande une ressource au serveur ;
2. le serveur répond au client avec un code 30X que la ressource est disponible à une autre adresse et lui fourni l'adresse,
3. le client consulte la nouvelle adresse.

Les manipulations de l'entête en PHP se font avec `header()`¹⁷, l'en-tête à modifier sera la "Location:".

La liste des utilisateurs est définie par le tableau :

```
$users = array(
    "jojo" => array("password" => "pass1", "status" => "administrator"),
```

¹³<https://www.php.net/manual/fr/function.setcookie.php>

¹⁴<https://www.php.net/manual/fr/reserved.variables.cookies.php>

¹⁵<https://www.php.net/manual/fr/function.htmlentities.php>

¹⁶<https://developer.mozilla.org/fr/docs/Web/HTTP/Redirections>

¹⁷<https://www.php.net/manual/fr/function.header.php>

```
"raoul" => array("password" => "pass2", "status" => "visitor"),
"roméo" => array("password" => "pass3", "status" => "customer"),
);
```

La fonction `array_key_exists()` peut vous aider mais n'est pas indispensable¹⁸.

5.2.2.2. Utiliser une variable de session

Maintenant que nous avons pu nous authentifier, nous allons créer une variable de session¹⁹, dans cette variable de session vous stockerez le pseudo et le status puis vous effectuerez une direction vers `site.php`.

Si la variable de session n'est pas positionnée alors `site.php` devra afficher accès refusé sinon le pseudo sera affiché et en fonction du status différents affichage :

visitor

consulter

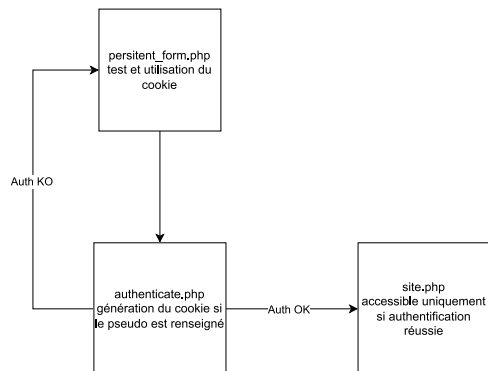
customer

consulter, acheter

administrator

consulter, acheter, administrer

Figure 2.9. Session



6. Pour aller plus loin : Limitons les accès

Nous avons un site sans authentification où tout utilisateur peut modifier, supprimer, créer un produit. Vous allez en utilisant une variable de session restreindre les modifications aux seuls administrateurs authentifiés.

Une table utilisateur avec les mots de passe encryptés :

```
CREATE TABLE IF NOT EXISTS user (
  pseudo TEXT PRIMARY KEY,
  password TEXT NOT NULL,
  status TEXT NOT NULL
  CHECK ( status IN ('Administrator','Customer','Visitor'))
);
/* password = pass */
INSERT OR REPLACE INTO user (pseudo,password,status)
VALUES ('Jojo',
'$2y$10$JMuaDMCavASPkf9KBcd1eaMHJ0zkeD8eYs7HjecoD8QeUVRhKQq6',
'Visitor');
INSERT OR REPLACE INTO user (pseudo,password,status)
```

¹⁸<https://www.php.net/manual/fr/function.array-key-exists.php>

¹⁹<https://www.php.net/manual/fr/session.examples.basic.php>


```
VALUES ('Raoul',
'$2y$10$JMuuaDMCavASPKf9KBcD1eaMHJ0zkeD8eYs7HjecoD8QeUVRhKQq6',
'Customer');
INSERT OR REPLACE INTO user (pseudo,password,status)
VALUES ('Romeo',
'$2y$10$JMuuaDMCavASPKf9KBcD1eaMHJ0zkeD8eYs7HjecoD8QeUVRhKQq6',
'Administrator');
```

Une entité UserEntity avec trois méthodes spécifiques : isValidPassword(string \$password):bool qui vérifie si un mot de passe correspond à celui encrypté, setPassword(bool|string|null \$password): void qui encrypte le mot de passe, setEncryptedPassword(bool|string|null \$password): void qui ne l'encrypte pas :

```
<?php
class UserEntity
{
    private string $pseudo;
    private string $password;
    private string $status;

    public function isValidPassword(string $password):bool {
        return password_verify($password, $this->password);
    }

    /**
     * @return string
     */
    public function getPseudo(): string
    {
        return $this->pseudo;
    }

    /**
     * @param string $pseudo
     */
    public function setPseudo(string $pseudo): void
    {
        $this->pseudo = $pseudo;
    }

    /**
     * @return false|string|null
     */
    public function getPassword(): bool|string|null
    {
        return $this->password;
    }

    /**
     * @param false|string|null $password
     */
    public function setPassword(bool|string|null $password): void
    {
        $this->password =password_hash($password, PASSWORD_DEFAULT);
    }

    /**
     * @param false|string|null $password
     */
    public function setEncryptedPassword(bool|string|null $password): void
    {
        $this->password = $password;
    }

    /**
     * @return string
     */
}
```

```

public function getStatus(): string
{
    return $this->status;
}

/**
 * @param string $status
 */
public function setStatus(string $status): void
{
    $this->status = $status;
}
}

```

Un dépôt similaire à celui des produits :

```

<?php

class DbUserRepository implements UserRepositoryInterface
{
    private $connexion;
    public function __construct()
    {
        $dsn = "sqlite:".CFG["db"]["host"].CFG["db"]["database"];
        $this->connexion = PDO::getInstance($dsn,
            CFG["db"]["login"], CFG["db"]["password"],
            CFG["db"]["options"], CFG["db"]["exec"])
            ->getConnexion();
    }

    public function findAll(): array
    {
        $statement = $this->connexion->prepare("SELECT * FROM user");
        $statement->execute();
        return $statement->fetchAll(PDO::FETCH_CLASS, "UserEntity");
    }

    public function findByPseudo(string $pseudo): ?UserEntity
    {
        $statement = $this->connexion->prepare("select *
            from user where pseudo=:pseudo");
        $statement->bindParam('pseudo', $pseudo);
        $statement->execute();
        $res = $statement->fetch(PDO::FETCH_ASSOC);
        $user = null;
        if ($res) {
            $user = new UserEntity();
            $user->setPseudo($res["pseudo"]);
            $user->setEncryptedPassword($res["password"]);
            $user->setStatus($res["status"]);
        }
        return $user;
    }

    public function add(UserEntity $user): ?UserEntity
    {
        $pseudo = $user->getPseudo();
        $password = $user->getPassword();
        $status = $user->getStatus();
        try {
            $statement = $this->connexion
                ->prepare("insert into user values(:pseudo,:password,:status)");
            $statement->bindParam('pseudo', $id);
            $statement->bindParam('password', $password);
            $statement->bindParam('status', $status);
            $statement->execute();
        } catch (Exception $e){}
        $user = $this->findByPseudo($pseudo);
    }
}

```

```

        return $user;
    }

    public function delete(string $pseudo): bool
    {
        $statement = $this->connexion->prepare("delete
            from user where pseudo=:pseudo");
        $statement->bindParam('pseudo', $pseudo);
        $res = $statement->execute();
        return $res;
    }

    public function update(string $pseudoS, UserEntity $user): ?UserEntity
    {
        $pseudo = $user->getPseudo();
        $password = $user->getPassword();
        $status = $user->getStatus();

        try{
            $statement = $this->connexion->prepare("
                update user
                set pseudo=:pseudo, password=:password, status=:status
                where pseudo=:pseudoS");
            $statement->bindParam('pseudoS', $pseudoS);
            $statement->bindParam('pseudo', $pseudo);
            $statement->bindParam('password', $password);
            $statement->bindParam('status', $status);
            $statement->execute();
        } catch (Exception $e){}
        $user = $this->findByPseudo($user);
        return $user;
    }
}

```

Un contrôleur à compléter :

```

<?php
namespace app\controller;

class User
{
    private \app\model\repository\UserRepositoryInterface $repository;
    public function __construct()
    {
        $this->repository = new \app\model\repository\DbUserRepository();
    }

    function login(){
        require "view".DIRECTORY_SEPARATOR."login.php";
    }

    function loginCheck(){

        $pseudo = $_POST['pseudo'];
        $password = $_POST['password'];
        $user = $this->repository->findByPseudo($pseudo);

        if ($user != null && $user->isValidPassword($password)) {
            //TODO
            die();
        }
        //TODO
    }

    function logout(){
        //TODO
    }
}
}D

```

Chapter 3. Création d'un micro-framework

//TODO

1. L'approche MVC

Le patron de conception ou motif d'architecture logiciel MVC (Modèle Vue Contrôleur) permet de séparer la logique métiers de l'affichage, il est composé de trois parties :

Le modèle

qui gère les données et la logique métiers

Le contrôleur

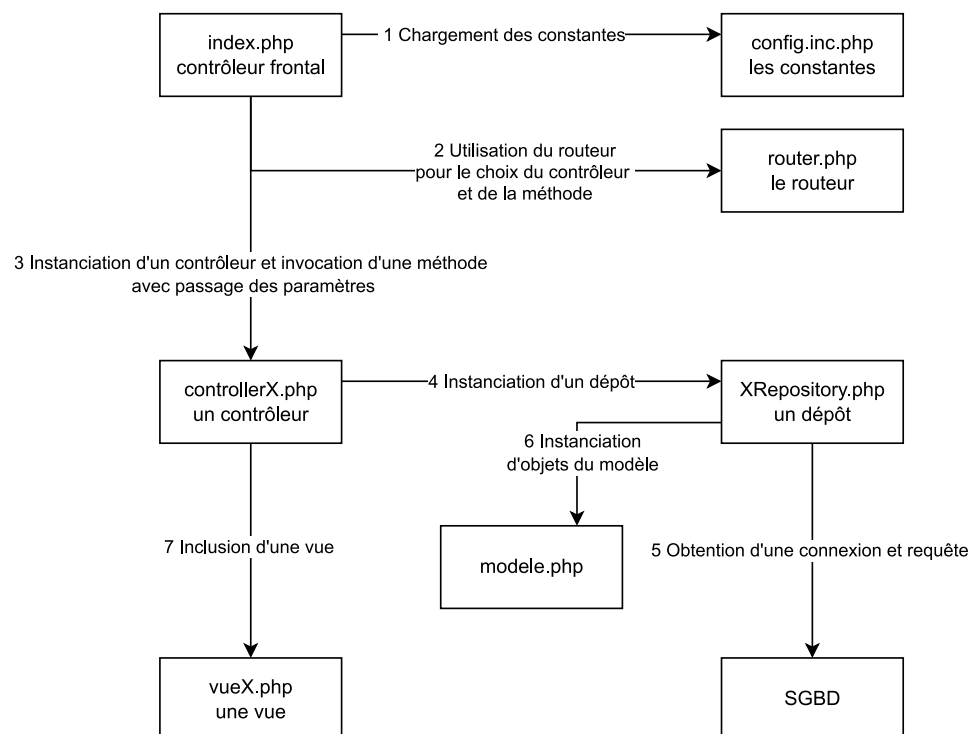
qui gère les demandes utilisateur en sollicitant le modèle et en modifiant la vue

La vue

qui est l'interface l'utilisateur

Nous allons réaliser une implémentation du MVC adaptée au web.

Figure 3.1. Notre MVC



1.1. Le contrôleur frontal et l'inclusion des constantes

Le contrôleur frontal sera le seul fichier PHP qui pourra être appelé directement, les autres seront inclus depuis celui-ci. Créer ou récupérer l'arborescence suivante sur git :

```
- td2
- app
```

```
-config
  -config.inc.php
  -index.php
  -system
```

app contiendra notre application et system des utilitaires.

Nous allons commencer par forcer l'appel du contrôleur frontal, apache permet de redéfinir localement sa configuration avec un fichier `.htaccess`, ce fichier redéfinit la configuration sur le répertoire qui le contient et ceux qui sont inclus. Voici le mien que vous devrez placer dans `td2/app` et adapter avec votre url-locale *L'url relative est celle qui suit le nom d'hôte et le port* (FallbackResource url-locale) :

```
RewriteEngine On

FallbackResource /~jub/mvc/app/index.php
```

`RewriteEngine On` active le module de réécriture d'apache utile pour la directive `FallbackResource`¹, cette directive indique que si un fichier n'est pas trouvé alors `index.php` est servi. Tester avec des URL incluses dans `td2`.

Nous allons depuis `index.php` inclure `config.inc.php` qui contiendra la définition de nos constantes. Pour inclure un autre script vous devez utiliser une des structures de contrôle suivante : `include`, `include_once`, `require`, `require_once`². Il existe également des possibilités d'auto-chargement, nous les utiliserons bientôt dans cette partie³.

Dans `config.inc.php` définir avec `define`⁴ la constante `HOME` pour qu'elle contienne le chemin absolu avec le répertoire `td2/app`. Vous devrez utiliser la constante magique `__DIR__`⁵, la concaténation qui est le `.` en PHP et la constante pré-définies `DIRECTORY_SEPARATOR`. Faites afficher cette constante dans `index.php`. Vous devez obtenir l'affichage d'un chemin absolu similaire à celui-ci

```
/chemin_vers_td2/td2/app/config/..
```

Les inclusions sont calculées à partir du premier fichier chargé, `index.php` pour nous. Pour faciliter les inclusions nous allons définir un tableau constant pour stocker l'URL du site et les informations sur la base de données :

```
const CFG = array(
    "db" => array(
        "host" => HOME."data".DIRECTORY_SEPARATOR,
        "port" => null,
        "database" => "madb.db",
        "login" => "",
        "password" => "",
        "options" => array(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION),
        "exec" => "PRAGMA foreign_keys = ON;"
    ),
    "siteURL" => "http://localhost/~jub/php/td2/app/" //votre url
);
```

Nous utiliserons SQLite un moteur SQL léger qui peut-être exécuté en local.

Une fois les constantes de l'application définies, notre contrôleur frontal va faire appel à un routeur.

1.2. Le routage

Nous avons vu que dans l'architecture un client effectue une requête dans l'attente d'une réponse. Le routeur a pour objectif d'analyser la requête pour déclencher l'exécution d'une méthode d'un contrôleur avec d'éventuels paramètres.

Le format d'URL retenu est `/contrôleur/méthode/param1/param2`. Si une méthode n'est pas fournie alors la méthode `index` sera choisie. Par exemple `/Home` déclenche la méthode `index` du contrôleur `Home`, `/User/add/jojo/44` déclenche la méthode `add` du contrôleur `User` avec les paramètres `jojo` et `44`.

¹https://httpd.apache.org/docs/trunk/fr/mod/mod_dir.html#fallbackresource

²<https://www.php.net/manual/fr/function.include.php>

³<https://www.php.net/manual/fr/language.oop5.autoload.php>

⁴<https://www.php.net/manual/fr/function.define.php>

⁵<https://www.php.net/manual/fr/language.constants.magic.php>

Nous allons créer une classe sans attribut ce qui peut-être contestable mais qui laisse la possibilité d'évolution comme par exemple de choisir les routes depuis un fichier de configuration, une route étant une association entre une URL et l'exécution d'une méthode d'un contrôleur.

Créer dans system une classe Router dans Router.php⁶.

```
-td2
-system
-Router.php
```

```
<?php
namespace system;

/**
 * Class Router
 */
class Router
{
    /**
     * permet d'exécuter la méthode d'un contrôleur choisi
     * si la méthode est omise alors index est choisi
     * Les URL saisies sont de la forme
     *     site/contrôleur
     *     site/contrôleur/method
     *     site/contrôleur/method/param1/...
     */
    function route()
    {
        $scriptName = $_SERVER["SCRIPT_NAME"];
        $requestURI = $_SERVER["REQUEST_URI"];

        //Le script name contient index.php on le supprime
        $prefix = substr($scriptName, 0, strlen($scriptName) - 9);
        //Le suffixe est le requestURI privé du préfix
        $suffix = substr($requestURI, strlen($prefix));

        $params = explode("/", $suffix);

        if (count($params) == 0) {
            echo "no controller found";
            die();
        }

        $controller = $params[0];
        array_shift($params);

        if (count($params) == 0)
            $controllerMethod = "index";
        else
            $controllerMethod = $params[0];

        array_shift($params);
        $class = "\\app\\controller\\" . $controller;
        $controllerInstance = new $class();
        $controllerInstance->$controllerMethod($params);
    }
}
```

Pour que nos tests soient efficaces, nous allons également créer un contrôleur qui sera choisi par le routage :

```
-td2
-app
-controller
-Home.php
```

```
class Home
```

⁶<https://www.php.net/manual/fr/language.oop5.php>

```
namespace app\controller;
{
//A terme, il n'y aura pas de echo dans un contrôleur
//Les echos seront dans les vues

    function index(){
        echo "index";
    }
    function method(){
        echo "method";
    }
    function methodWithParameter(array $params){
        var_dump($params);
    }
}
```

Si \$controller est le nom de votre contrôleur; \$controllerMethod celui de la méthode et \$params le tableau de paramètres alors le code suivant permet d'instancier un contrôleur et d'appeler la méthode.

```
$class = "\app\controller\\".$controller;
    $controllerinstance = new $class();
    $controllerinstance->$controllerMethod($params);
```

Le routeur sera instancié dans index.php et la méthode route sera invoquée :

```
$router = \system\Router();
$router->route();
```

Nous allons ici utiliser le chargement automatique des classes, lors de l'instanciation d'un objet, PHP chargera automatiquement la classe, placer le code suivant dans votre index.php après l'inclusion de config.inc.php et avant l'instanciation du routeur ⁷:

```
spl_autoload_register(function($class) {
    $path=$class;
    if (DIRECTORY_SEPARATOR === '/') {
        $path=str_replace('\\', '/', $class);
    }
    $include = HOME.".".DIRECTORY_SEPARATOR.$path.'.php';
    if (file_exists($include))
        require $include;
    else
        echo '<pre>'.$class.PHP_EOL.$include.PHP_EOL.'not found'. '</pre>';
});
```

Important

Cet autoload est minimaliste et présuppose que les instanciations soient fait à partir du namespace racine, les use ne peuvent-être utilisés. new \system\Router(); fonctionne, use system; new Routeur() ne fonctionne pas.

A ce stade, vous pouvez tester pour vérifier si les méthodes de vos contrôleurs sont bien appelées.

1.3. Les contrôleurs

Nous venons de voir que nos contrôleurs sont invoqués par le routeur après analyse de l'url. Les contrôleurs vont solliciter les modèles pour obtenir, créer, modifier des informations avant d'appeler des vues. Nous allons afficher ces étapes avant de les implémenter réellement :

```
namespace app\controller;
class Home
```

⁷Avec des espaces de nommage et une convention pour les répertoires, l'autoload peut-être simplifiée.

```
{
    //Création d'un attribut pour accéder au modèle
    public function __construct()
    {
        //instanciation de l'attribut
    }

    function index(){
        //Solicitation du modèle
        //Solicitation de la vue
    }
}
```

1.4. Le modèle

Nous allons utiliser le patron de conception dépôt ou repository, le repository est sollicité par le contrôleur, il sollicite la source de donnée et renvoie des entités. Commençons par créer une interface :

```
-td2
- app
- model
- repository
- ProductRepositoryInterface.php
```

```
<?php
namespace app\model\repository;
Interface ProductRepositoryInterface
{
    public function findAll(): array;
    public function findById(int $id):
        ?\app\model\entity\ProductEntity;
    public function add(\app\model\entity\ProductEntity $product):
        ?\app\model\entity\ProductEntity;
    public function update(int $id, \app\model\entity\ProductEntity $product):
        ?\app\model\entity\ProductEntity;
    public function delete(int $id): bool;
}
```

Nous allons également créer notre entité, elle ne contient que des attributs privés avec des getters, des setters et un constructeur sans paramètre⁸:

```
-td2
- app
- model
- entity
- ProductEntity.php
```

```
namespace app\model\entity;
class ProductEntity
{
    private int $id;
    private string $name;
    private float $price;
    private int $quantity;

    /**
     * @return int
     */
    public function getId(): int
    {
        return $this->id;
    }
}
```

⁸En PHP toute classe possède un constructeur par défaut qui est sans paramètre.


```

/**
 * @return string
 */
public function getName(): string
{
    return $this->name;
}

/**
 * @return float
 */
public function getPrice(): float
{
    return $this->price;
}

/**
 * @return int
 */
public function getQuantity(): int
{
    return $this->quantity;
}

/**
 * @param int $id
 */
public function setId(int $id): void
{
    $this->id = $id;
}

/**
 * @param string $name
 */
public function setName(string $name): void
{
    $this->name = $name;
}

/**
 * @param float $price
 */
public function setPrice(float $price): void
{
    $this->price = $price;
}

/**
 * @param int $quantity
 */
public function setQuantity(int $quantity): void
{
    $this->quantity = $quantity;
}
}

```

A vous de créer une classe `MemoryProductRepository` dans `repository` qui implémente `ProductRepositoryInterface` et qui dans `findAll()` renvoie un tableau de `ProductEntity`, le code des autres méthode reste vide :

```

<?php
namespace app\model\repository;
class MemoryProductRepository implements ProductRepositoryInterface
{
    public function findAll(): array
    {
        //TODO
    }
}

```

```
//TODO
}
```

Modifier votre contrôleur Home pour que l'attribut soit de type `ProductRepositoryInterface` et lui affecter dans le constructeur une instance de `MemoryProductRepository` enfin dans la méthode `index` appeler la méthode `findAll()` du repository et vérifier le résultat avec un `var_dump()`.

1.5. La vue

A ce stade, notre contrôleur dispose des données du modèle et souhaite les afficher, c'est la mission de la vue. Nous pourrions comme pour le Router créer une classe mais dans un soucis de simplification nous allons directement inclure la page web. Avec une inclusion, la page aura accès à l'ensemble des variables disponibles dans le contrôleur donc aux données qui viennent du modèle ce qui est souhaité mais la page aura également accès aux attributs du contrôleur, ce qui n'est pas souhaitable. Créer une vue et y faire afficher vos produits.

```
-td2
  -app
    -view
      -home.php
```

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon magasin</title>
</head>
<body>
<table>
  <thead>
    <tr>
      <th> id </th>
      <th> name </th>
      <th> price </th>
      <th> quantity </th>
    </tr>
  </thead>
  <tbody>
    <?php ?>
    <tr>
      <td> <?= ?> </td>
      <td> <?= ?> </td>
      <td> <?= ?> </td>
      <td> <?= ?> </td>
    </tr>
    <?php ?>
  </tbody>
</table>
</body>
</html>
```

2. La persistance

Nous avons écrit beaucoup de code, pour au final produire un site statique, en effet, il renvoie toujours les mêmes informations. Les exécutions des scripts étant indépendantes, nous ne pouvons pas par exemple ajouter ou supprimer des produits. Nous allons utiliser un moteur base de données de pour assurer la persistance de nos données entre deux exécutions puis nous rajouterons des fonctionnalités à notre application.

2.1. Création et peuplement de la base

Nous allons utiliser SQLite pour créer la base `madb.db` dans `data` :

```
-td2
```

```
-app
  -data
    -madb.db
    -create_and_populate_madb.sql
```

Commençons par créer un script de création et de peuplement de la base de données (create_and_populate_madb.sql) :

```
CREATE TABLE IF NOT EXISTS product (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    price FLOAT NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 0
);

INSERT OR REPLACE INTO product (id,name,price, quantity) VALUES (1,'p1',10.0,1);
INSERT OR REPLACE INTO product (id,name,price, quantity) VALUES (2,'p2',20.0,1);
```

Pour exécuter ce script, nous allons utiliser la commande `sqlite3`⁹.

Pour exécuter notre script de création depuis data utiliser : `sqlite3 madb.db '.read create_and_populate_madb.sql'`.

Vous connecter sur la base (`sqlite3 madb.db`), vérifier les tables (`.table`), vérifier le schéma (`.schema`), afficher le contenu (`select`), ajouter un nouvel enregistrement (`insert`), modifier le nouvel enregistrement (`update`), le supprimer (`delete`). Il faut également que le serveur apache dispose des droits d'écriture et de modification dans /app/data.

2.2. Création d'un repository avec une source de données

Nous allons commencer par créer un singleton qui nous permettra d'obtenir une instance de PDO et via cette instance d'obtenir une connexion :

```
-td2
  -system
    -SPDO.php
```

```
<?php
namespace system;

class SPDO
{
    private \PDO $connexion;
    private static ?SPDO $PDOInstance=null;

    private function __construct(string $dsn,
                                ?string $username = null,
                                ?string $password = null,
                                ?array $options = null,
                                ?string $exec = null)
    {
        $this->connexion = new \PDO($dsn,$username , $password, $options);
        $this->connexion->exec($exec);
    }

    public static function getInstance(string $dsn,
                                ?string $username = null,
                                ?string $password = null,
                                ?array $options = null,
                                ?string $exec = null):\system\SPDO
    {
        if(!self::$PDOInstance)
        {
            self::$PDOInstance = new \system\SPDO($dsn,
                $username , $password, $options, $exec);
        }
    }
}
```

⁹<https://www.sqlite.org/cli.html>

```

    }
    return self::$PDOInstance;
}

public function getConnexion():\PDO{
    return $this->connexion;
}
}

```

Cette classe possède un constructeur privé et un new est donc exclus, l'instanciation reposera sur l'utilisation de la méthode statique getInstance() qui renverra une nouvelle instance de PDO si elle n'a pas déjà été créée ou le cas échéant l'instance déjà créée. L'appel d'une méthode statique est réalisée via ::, n'oubliez pas ce que vous avez défini dans config.inc.php. Dans index.php obtenir une connexion et la faire afficher, le dsn sera au format sqlite:chemin/madb.db¹⁰.

Le front contrôleur n'est pas le bon endroit pour obtenir la connexion, en effet toutes les pages n'en n'auront pas besoin. Créer une classe DbProductRepository qui implémente ProductRepositoryInterface et qui a comme attribut une connexion (déplacer le code précédemment créé).

```

<?php

namespace app\model\repository;
class DbProductRepository implements ProductRepositoryInterface
{
    private $connexion;

    public function __construct()
    {
        $dsn = "sqlite:".CFG["db"]["host"].CFG["db"]["database"];
        $this->connexion = \system\SPDO::getInstance($dsn,
            CFG["db"]["login"],
            CFG["db"]["password"],
            CFG["db"]["options"],
            CFG["db"]["exec"])
            ->getConnexion();
    }

    public function findAll(): array
    {
        //TODO
    }

    public function add(\app\model\entity\ProductEntity $product):
        \app\model\entity\ProductEntity
    {
        //TODO
    }

    public function findById(int $id): ?\app\model\entity\ProductEntity
    {
        //TODO
    }

    public function update(int $idS, \app\model\entity\ProductEntity $product):
        \app\model\entity\ProductEntity
    {
        //TODO
    }

    public function delete(int $id): bool
    {
        //TODO
    }
}

```

Modifier le contrôleur Home pour qu'il utilise ce nouveau dépôt.

¹⁰<https://www.php.net/manual/fr/ref.pdo-sqlite.connection.php>

Enfin dans la méthode `findAll()` vous allez réaliser une requête préparée PDO : `prepare()`¹¹, `execute()`¹², `fetchAll()`¹³ avec le mode PDO : `FETCH_CLASS` et la classe `ProductEntity`.

Nous avons un site dynamique mais avec une seule fonctionnalité, nous allons y remédier.

3. Faisons grandir l'application

Nous allons ajouter des fonctionnalités : la création, la modification et la suppression d'un produit.

Commencer par créer un contrôleur `Produit` :

```
<?php
namespace app\controller;
class Product
{
    private \app\model\repository\ProductRepositoryInterface $repository;
    public function __construct()
    {
        $this->repository = new \app\model\repository\DbProductRepository();
    }

    function add(){
        require "view".DIRECTORY_SEPARATOR."addProduct.php";
    }

    function addProduct(){
        //TODO
    }

    function display( array $params){
        //$params est positionné par le routeur
        //comme étant un tableau indexé contenant les paramètres
        //TODO
    }

    function update(array $params){
        //TODO
    }

    function updateProduct(){
        //TODO
    }

    function delete(array $params){
        //TODO
    }
}
```

3.1. Afficher un produit

Nous allons afficher un produit si il existe et une erreur sinon.

Il nous faut une route :

`Product/display/id`

en GET qui pour afficher la vue du produit, elle conduit au contrôleur `Product` pour exécuter la méthode `display` avec comme paramètre un tableau qui contient l'id.

Il nous faut une nouvelle méthode du dépôt :

¹¹<https://www.php.net/manual/fr/pdo.prepare.php>

¹²<https://www.php.net/manual/fr/pdostatement.execute.php>

¹³<https://www.php.net/manual/fr/pdostatement.fetchall.php>

`findById(int $id):?ProductEntity`

qui renvoie le produit si il existe ou l'entité sinon.

Il nous faut également la vue

`displayProduct`

qui affiche le produit

Nous allons commencer par le modèle, en vous inspirant de `findAll`, créer une requête préparée pour afficher un produit. L'id est une entier reçu en paramètre, nous allons l'utiliser dans une requête préparée :

1. Utiliser `prepare`¹⁴ avec `:id` comme paramètre nommé
2. Réaliser l'association entre le paramètre effectif de la méthode (`$id`) et le paramètre nommé (`:id`) avec `bindParam`¹⁵.
3. Exécuter la requête.
4. Récupérer le résultat avec `fetch` en le `PDO::FETCH_ASSOC`.
5. Créer un objet de type `Product` si possible
6. Renvoyer le résultat.

Pour tester le modèle, rajouter la méthode `display` dans le contrôleur et tester avec plusieurs URL.

Quelques conseils :

- Tout ce qui vient du serveur est `string`¹⁶, `intval`¹⁷ et `floatval`¹⁸ permettent de transtyper.

3.2. Créer d'un produit

Pour créer un produit, il faut deux routes :

`Product/add`

en GET sans paramètre pour afficher le formulaire de création

`Product/addProduct`

en POST avec dans le corps `id`, `name`, `price`, `quantity` pour modifier le modèle.

Pour le dépôt, nous ajouterons une méthode :

`add(ProductEntityProduct $product):?ProductEntity`

qui reçoit un produit en paramètre, essaye de le créer puis le récupère dans la base

Pour les vues, nous aurons le formulaire de saisie :

`addProduct`

formulaire permettant en POST d'envoyer à `Product/addProduct`

`displayProduct`

pour afficher le produit créé ou une erreur si la création est impossible

¹⁴<https://www.php.net/manual/en/pdo.prepare>

¹⁵<https://www.php.net/manual/fr/pdostatement.bindparam>

¹⁶<https://www.php.net/manual/fr/language.types.string.php>

¹⁷<https://www.php.net/manual/fr/function.intval.php>

¹⁸<https://www.php.net/manual/fr/function.floatval>

3.3. Modifier un produit

Pour modifier vous devez modifier la vue home.php pour rajouter une colonne, cette colonne contiendra une ancre vers la route `Product/update/id`.

La méthode `update` sollicitera le modèle pour à partir de l'id récupérer le produit puis affichera la vue `updateProduct` qui contiendra un formulaire prérempli et l'id masqué. Ce formulaire conduira à la méthode `updateProduct` qui réalisera la mise à jour avant d'afficher le produit ajouté avec `Product/display/id`. Il faudra également ajouter une méthode `update(int $id, ProductEntity $product):ProductEntity` au dépôt, `$id` sera l'identifiant du produit à modifier et `$product` les nouvelles valeurs.

3.4. Supprimer un produit

Pour supprimer un produit vous devez modifier la vue home.php pour rajouter une colonne, cette colonne contiendra une ancre vers la route `Product/delete/id`. Dans le dépôt rajouter la méthode `delete(int $id): bool`. Une fois le modèle modifié, vous pouvez faire une direction vers `/Home`.

Chapter 4. Mise en oeuvre d'un framework (CI4)